Chapitre 15 : Algorithmes A*

ITC - 2025/2026

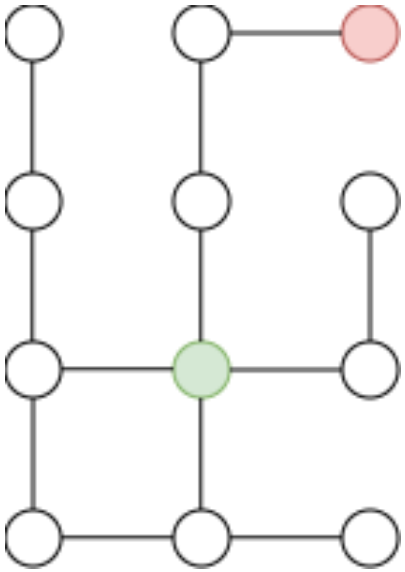
Félix POTIÉ

Problème du labyrinthe 3
Algorithme de Dijkstra 4
Heuristique 6
Choix de l'heuristique 8
Algorithme A* 13
Visualisation 26
Application : GPS 29

Problème du labyrinthe

Problème 1.

On est dans un labyrinthe, on cherche à en sortir.
On connaît les **coordonnées** de `src` le point de départ et `dst` la sortie du labyrinthe.
On cherche **le plus court chemin** entre `src` et `dst` en respectant les contraintes du labyrinthe.



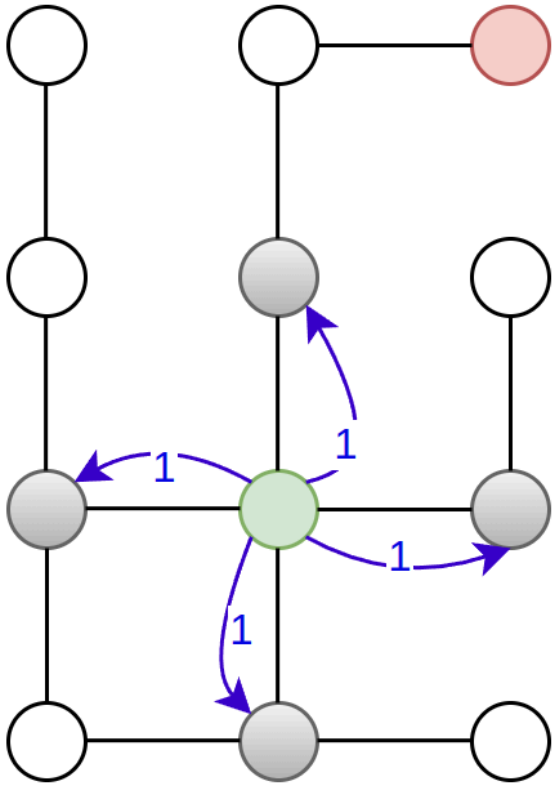
Graphe modélisant les passages ouverts d'un labyrinthe.

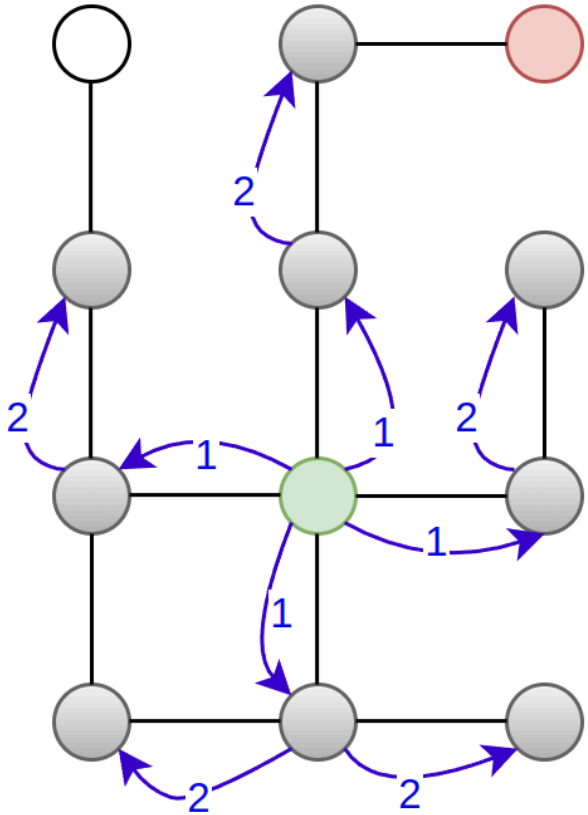
En vert le point de départ `src` et en rouge la sortie `dst`.

Chaque arête est de poids 1.

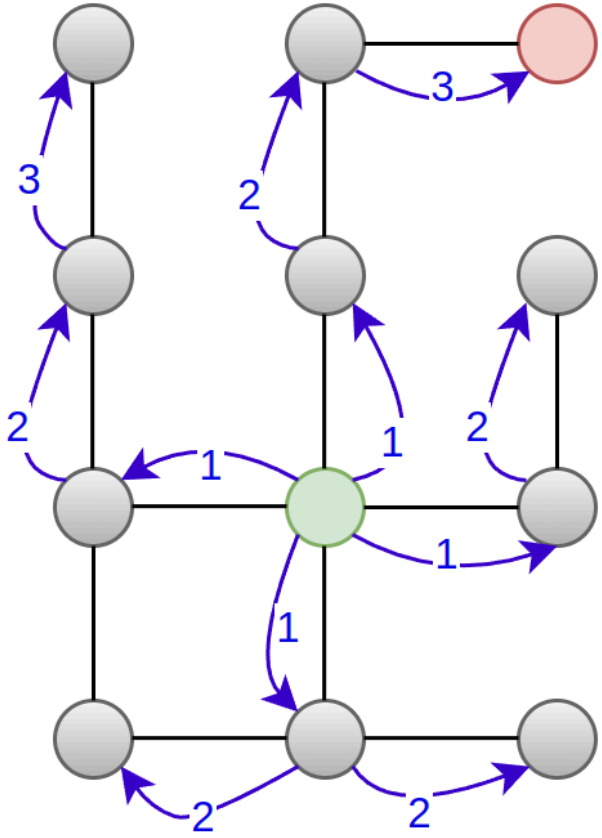
Algorithme de Dijkstra

On pourrait utiliser l'algorithme de Dijkstra pour résoudre ce problème (ou même un parcours en largeur).





Algorithme de Dijkstra - 2



En dépilant Dijkstra, on trouve le plus court chemin.

Cependant, on a exploré beaucoup de sommets qui ne menaient pas au plus court chemin.

Dans notre cas, on a même exploré tous les sommets (en fonction de l'ordre dans lequel on retire de la file les égalités).

Heuristique

Objectif : améliorer l'algorithme de Dijkstra pour qu'il explore moins de sommets.

Constat : avec notre algorithme de Dijkstra, on ne va pas toujours dans une direction qui nous rapproche de la sortie.
On possède les coordonnées de la sortie, on peut donc utiliser cette information pour guider notre algorithme.

Exemple : on ne va pas aller dans une direction opposée à la sortie.

Pour améliorer l'algorithme de Dijkstra, on peut utiliser une fonction d'heuristique.

Heuristique - 2

Définition 1: Fonction d'heuristique.

Dans le cadre des algorithmes de recherche, la fonction d'heuristique, notée généralement $h(n)$, est une fonction qui **estime** le "coût" restant pour atteindre l'objectif depuis un point donné.

$$h(n) = \text{coût estimé entre le sommet et l'objectif final}$$

Souvent, (quand on travaille sur des plans, des cartes, etc.) on utilise une distance à vol d'oiseau (euclidienne).

Choix de l'heuristique

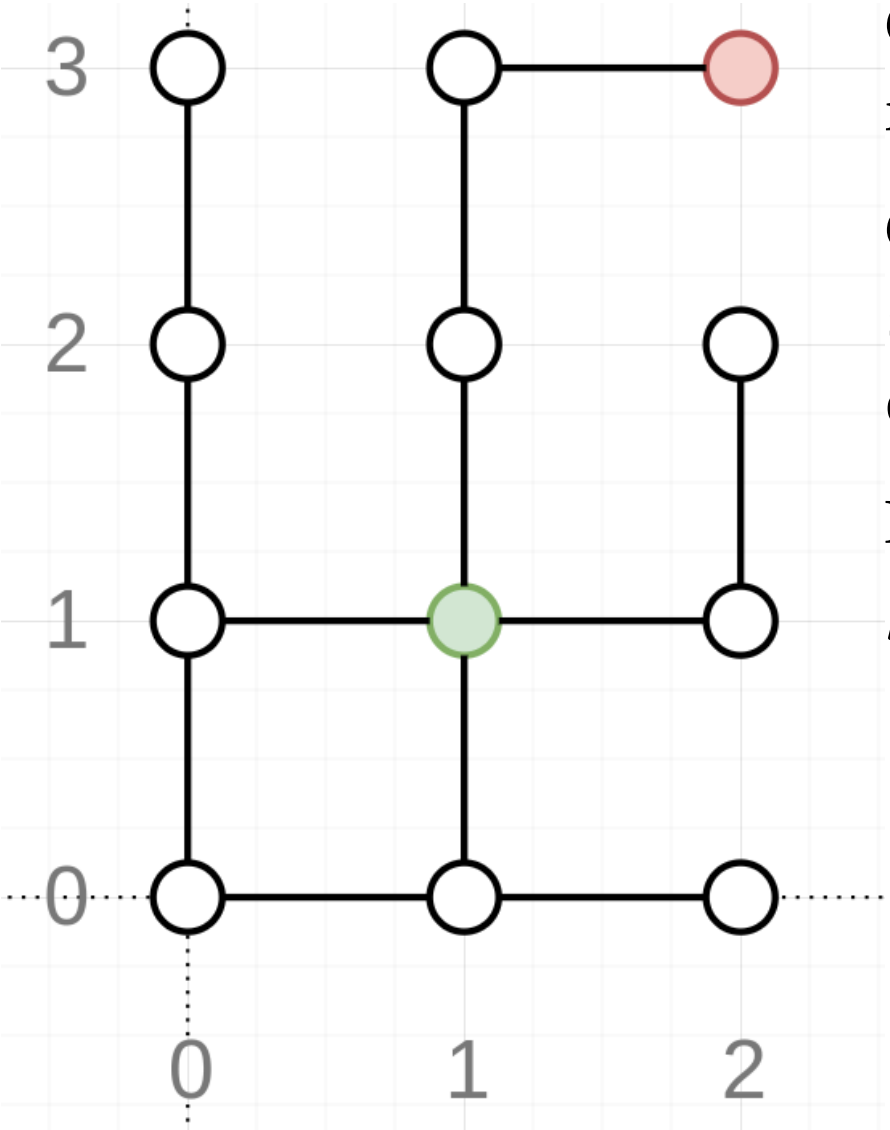
Dans le cas de notre labyrinthe, on peut utiliser la distance de Manhattan comme fonction d'heuristique.

Définition 2: Distance de Manhattan.

La distance de Manhattan entre deux points $a = (x_a, y_a)$ et $b = (x_b, y_b)$ est donnée par :

$$d(a, b) = |x_a - x_b| + |y_a - y_b|$$

Choix de l'heuristique - 2



On place notre graphe dans un repère orthonormé.

On a :

`src`$= (1, 1)$

`dst`$= (2, 3)$

Donc :

$$\begin{aligned} h(\text{src}) &= d(\text{src}, \text{dst}) \\ &= |1 - 2| + |1 - 3| \\ &= 1 + 2 \\ &= 3 \end{aligned}$$

Choix de l'heuristique - 3

Propriété 1: Heuristique admissible.

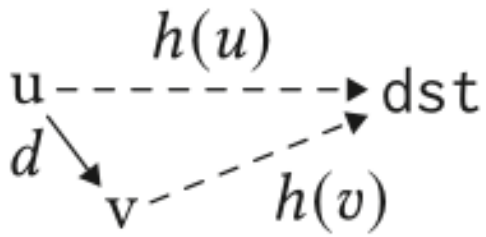
Pour que notre algorithme fonctionne, on choisit une heuristique **admissible**. C'est-à-dire une heuristique qui est une **sous-estimation** du coût réel. Autrement dit $h(n) \leq d(n)$ avec $d(n)$ le coût réel entre le sommet n et l'objectif final.

Choix de l'heuristique - 4

Propriété 2: Heuristique monotone.

La fonction h est dite monotone si pour tout arc $u \xrightarrow{d} v$ du graphe, on a l'inégalité $h(u) \leq d + h(v)$.

On peut visualiser cette propriété comme une forme d'inégalité triangulaire :



Avoir une heuristique monotone permet de garantir que la première fois qu'un sommet est extrait de la file de priorité, on a trouvé le plus court chemin pour l'atteindre.

Choix de l'heuristique - 5

On admet que la distance de Manhattan est **admissible** et **monotone**.

Algorithme A*

L'algorithme **A*** est une amélioration de l'algorithme de Dijkstra qui utilise une fonction d'heuristique pour guider la recherche du plus court chemin.

On pose l'heuristique totale : $ht(u) = h(u) + d(u)$ avec $d(u)$ le poids du chemin parcouru depuis le sommet *src* jusqu'à u.

Dans notre file de priorité, on va stocker des 3-uplets `(ht_u, d_u, u)` avec :

- `ht_u` : l'heuristique totale
- `d_u` : le poids du chemin depuis `src` jusqu'à `u`
- `u` : le sommet

On défilera donc le sommet `u` de la file de priorité avec `ht_u` minimal.

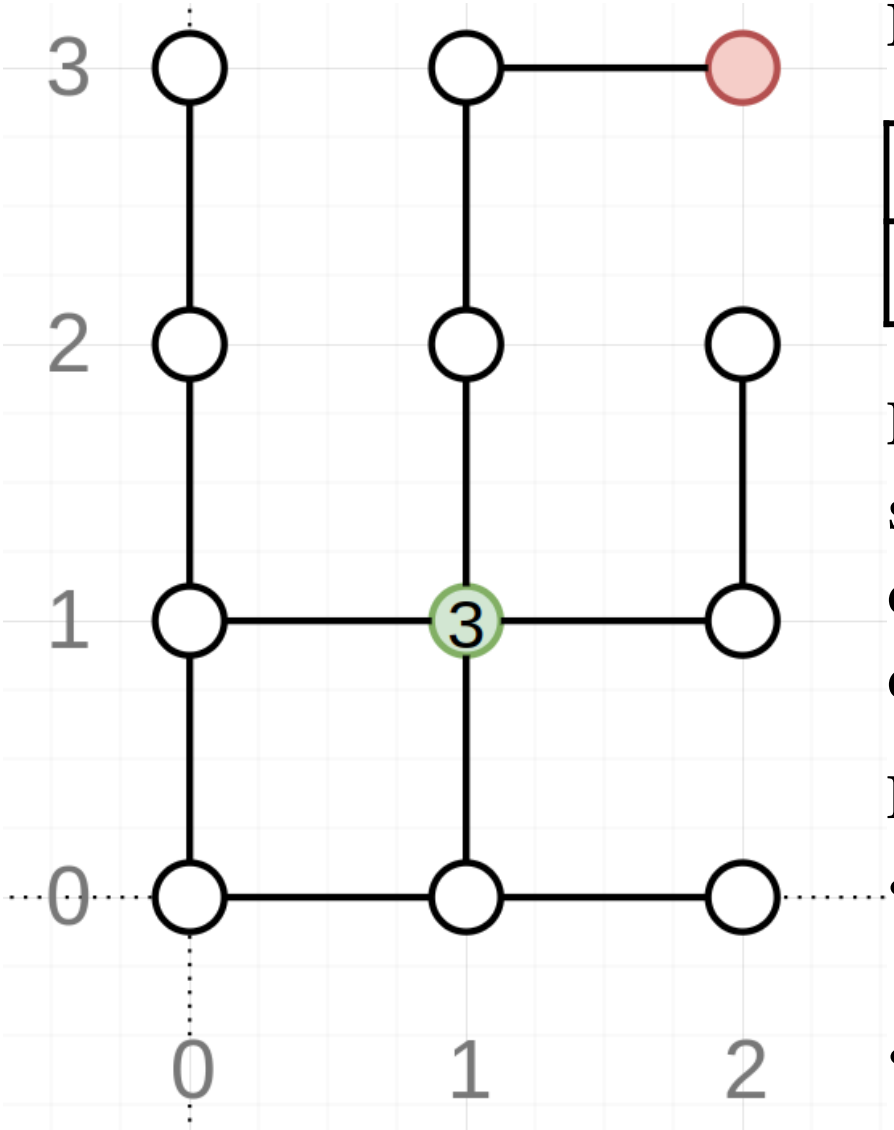
Algorithme A* - 2

Algorithme 1: Algorithme A*.

Entrée : G, `src`, `dst`, fonction d'heuristique h

- `f_prio` = file de priorité contenant `(`h`(src), 0, src)`
- `visites` = ensemble vide
- **Tant que** `f_prio` n'est pas vide :
  - `(ht_u, d_u, u)` = premier élément de `f_prio`
  - **Si** `u` est dans `visites` : **Continuer** (passer au suivant)
  - **Si** `u` = `dst` : **Retourner** `d_u`
  - ajouter `u` à `visites`
  - **Pour chaque** voisin `v` de `u` :
    - **Si** `v` n'est pas dans `visites` :
      - `d_v` = `d_u` + poids(u, v)
      - `ht_v` = `d_v` + $h(v)$
      - ajouter `(ht_v, d_v, v)` à `f_prio`

Algorithme A* - 3



Initialisation :

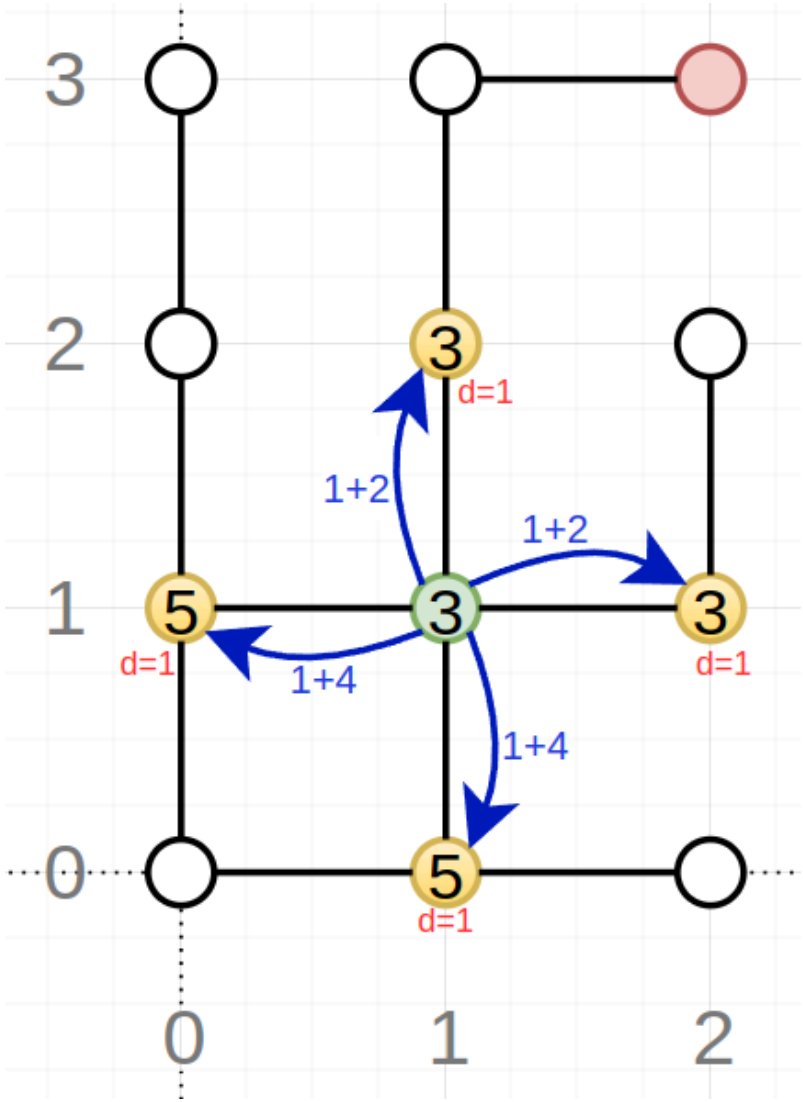
`f_prio`	`visites`
[(3, 0, `(1,1)`)]	{ }

Dans la suite on note sur les sommets l'heuristique totale et à côté des sommets le poids du chemin depuis src.

Les sommets :

- jaunes sont dans la file de priorité.
- bleus sont visités.

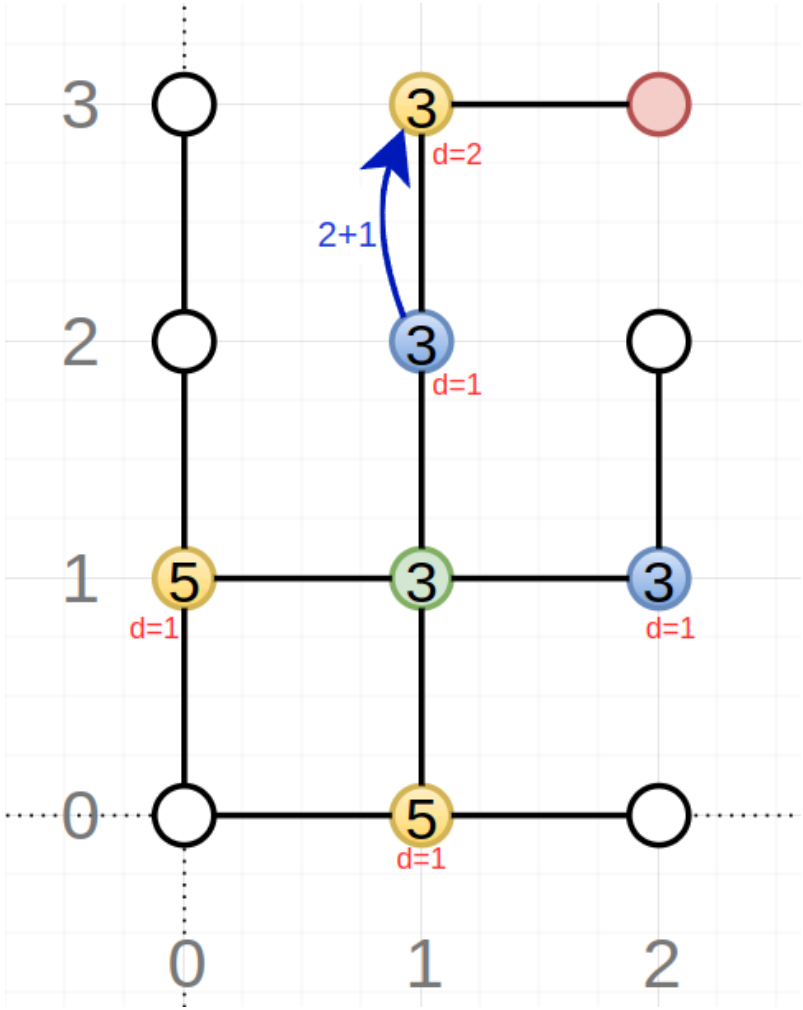
Algorithme A* - 4



On défile `(1,1)` et on enfile ses voisins avec les bonnes valeurs.

f_prio	**visites**
[(3, 1, `(1,2)`), (3, 1, `(2,1)`), (5, 1, `(0,1)`), (5, 1, `(1,0)`)]	{ `(1,1)` }

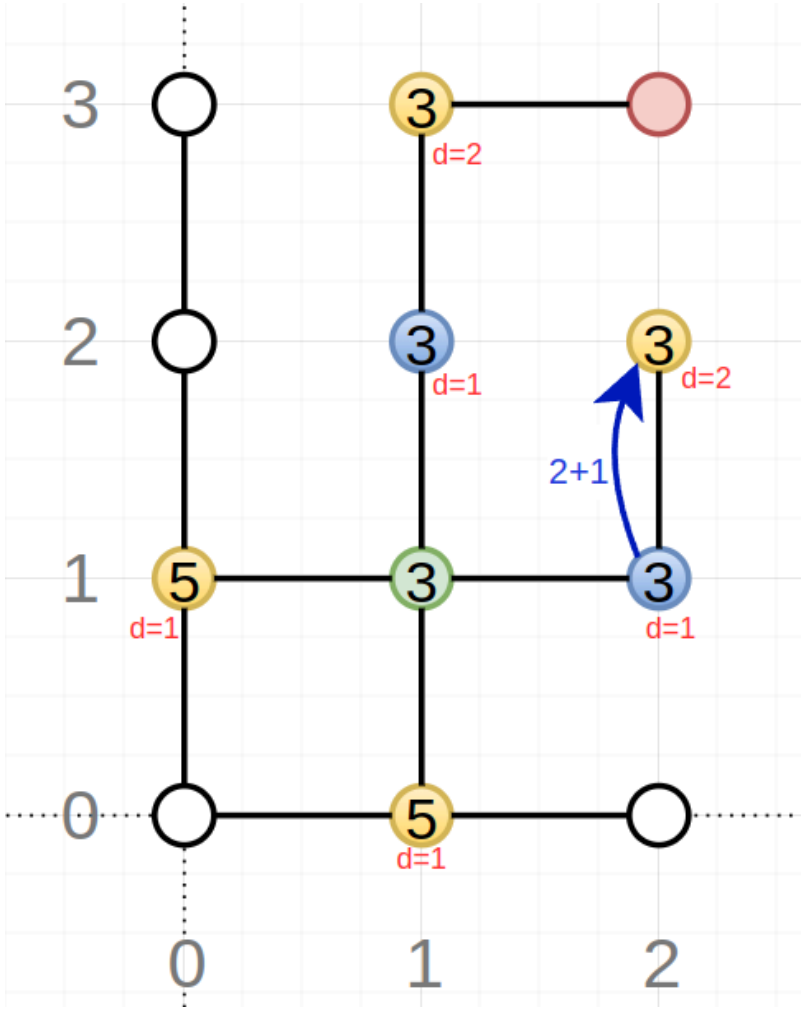
Algorithme A* - 5



On défile `(1,2)` et on enfile ses voisins (non visités) avec les bonnes valeurs.

f_prio	**visites**
[(3, 1, `(2,1)`), (3, 2, `(1,3)`), (5, 1, `(0,1)`), (5, 1, `(1,0)`)]	{ `(1,1)`, `(1,2)` }

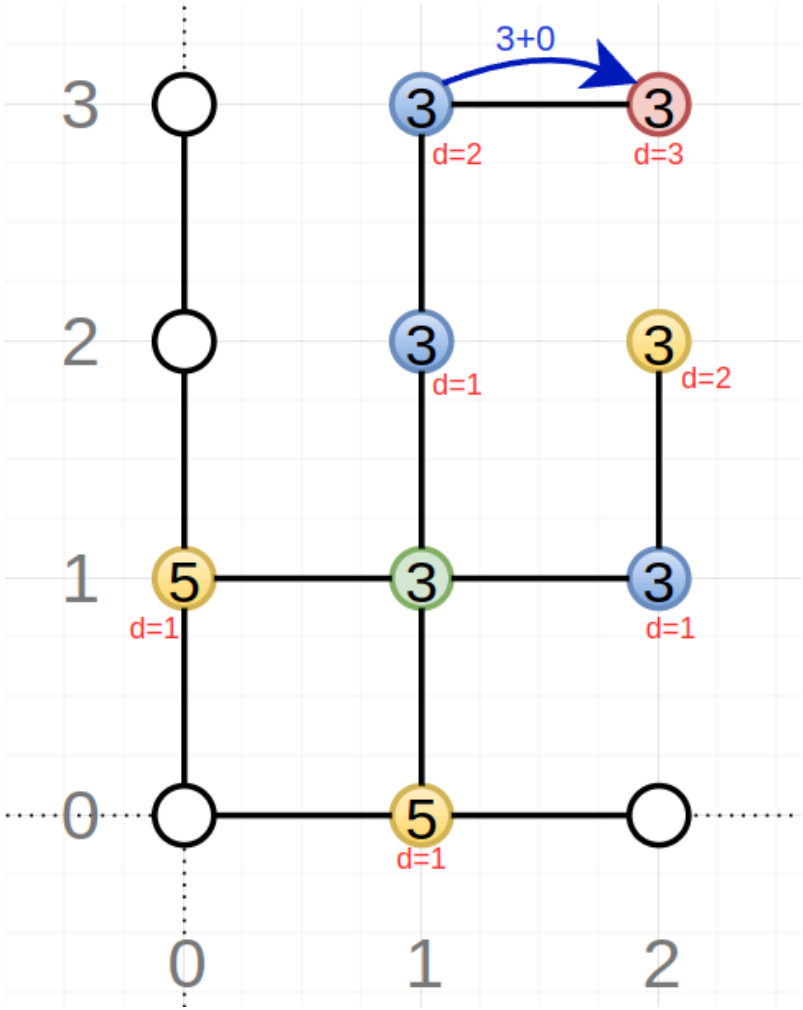
Algorithme A* - 6



On défile `(2,1)` et on enfile ses voisins (non visités) avec les bonnes valeurs.

f_prio	**visites**
[(3, 2, `(1,3)`), (3, 2, `(2,2)`), (5, 1, `(0,1)`), (5, 1, `(1,0)`)]	{ `(1,1)`, `(1,2)`, `(2,1)` }

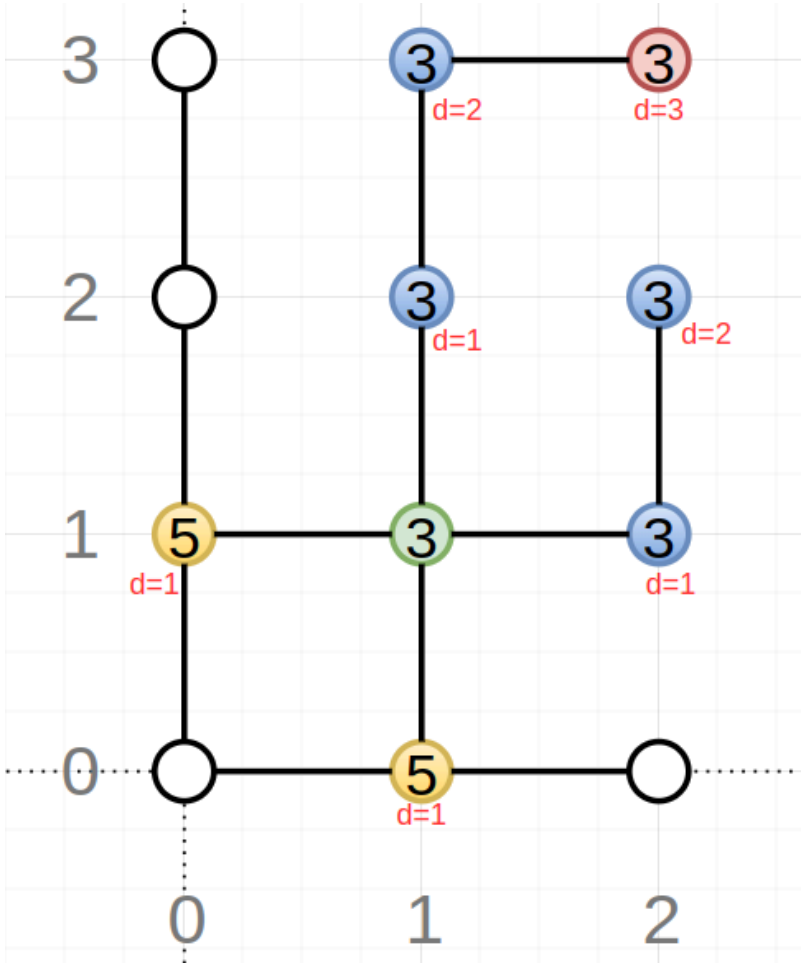
Algorithme A* - 7



On défile `(1,3)` et on enfile ses voisins (non visités) avec les bonnes valeurs.

f_prio	**visites**
[(3, 3, `(2,3)`), (3, 2, `(2,2)`), (5, 1, `(0,1)`), (5, 1, `(1,0)`)]	{ `(1,1)`, `(1,2)`, `(2,1)`, `(1,3)` }

Algorithme A* - 8

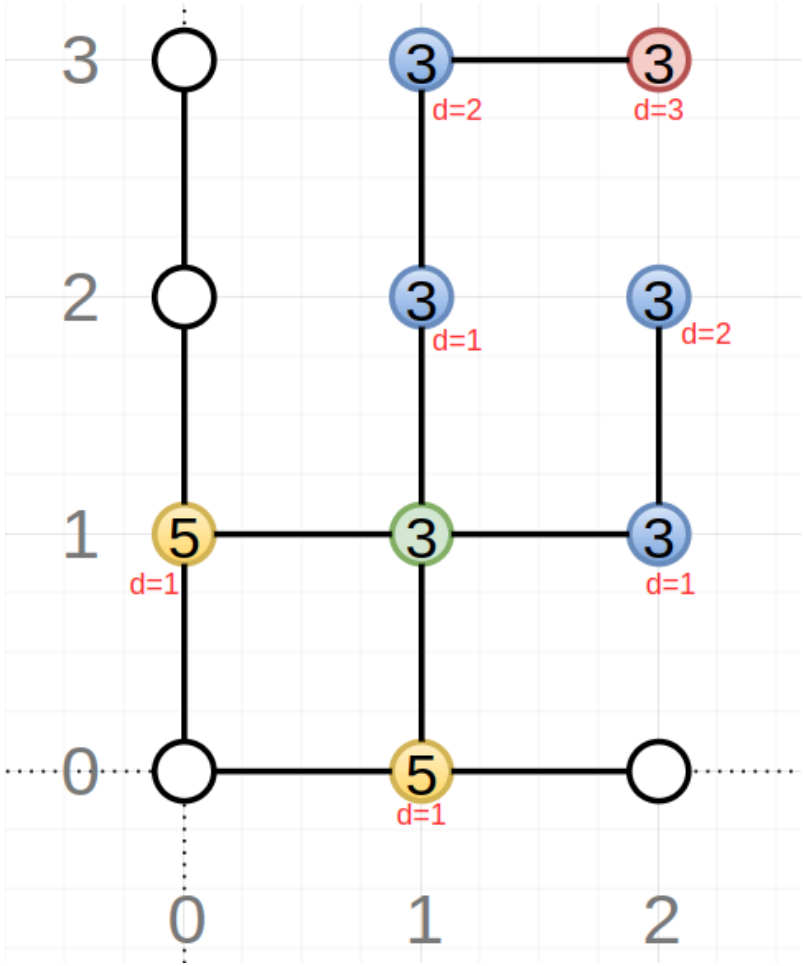


On défile `(2,2)`, il n'a pas de voisin non visité.

f_prio	**visites**
[(3, 3, `(2,3)`), (5, 1, `(0,1)`), (5, 1, `(1,0)`)]	{ `(1,1)`, `(1,2)`, `(2,1)`, `(1,3)`, `(2,2)` }

On a donc trouvé le plus court chemin sans avoir besoin d'explorer tous les sommets.

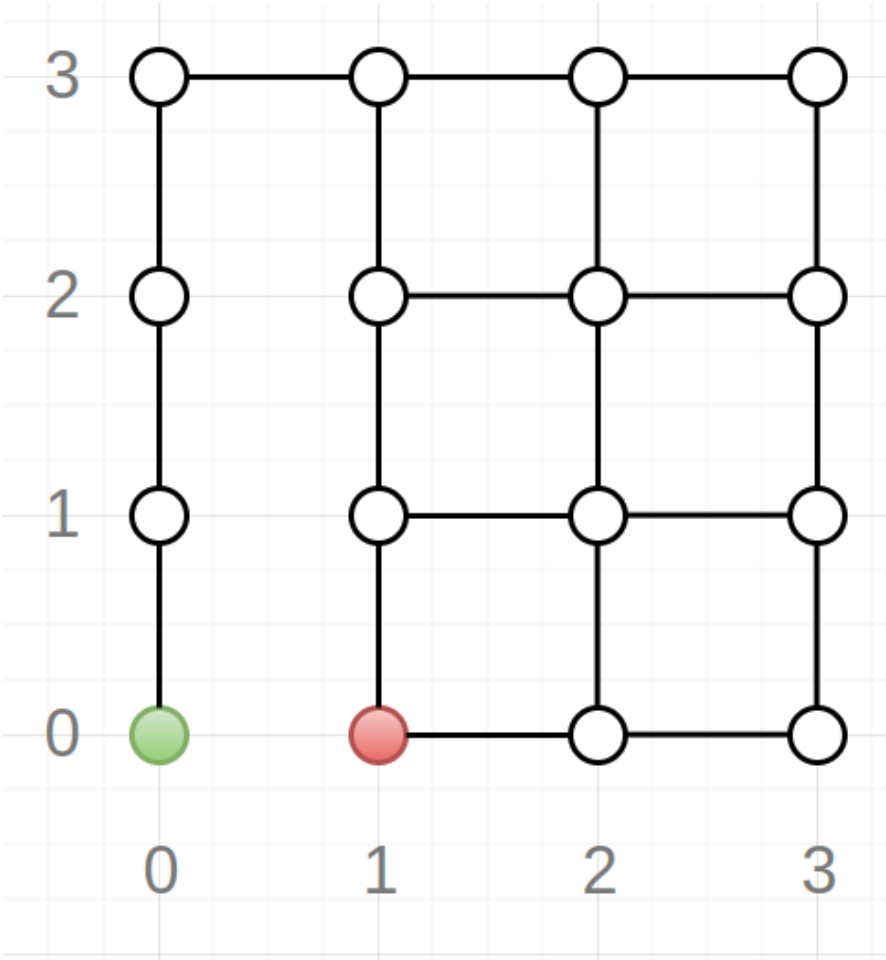
Algorithme A* - 9



On défile `(2,3)` qui est notre destination, on retourne donc `3`, la distance la plus courte de `src` à `dst`.

f_prio	**visites**
[(5, 1, `(0,1)`), (5, 1, `(1,0)`)]	{ `(1,1)`, `(1,2)`, `(2,1)`, `(1,3)`, `(2,2)` }

Algorithme A* - 10



Dépiler **Dijkstra** puis **A*** sur ce graphe.

Algorithme A* - 11

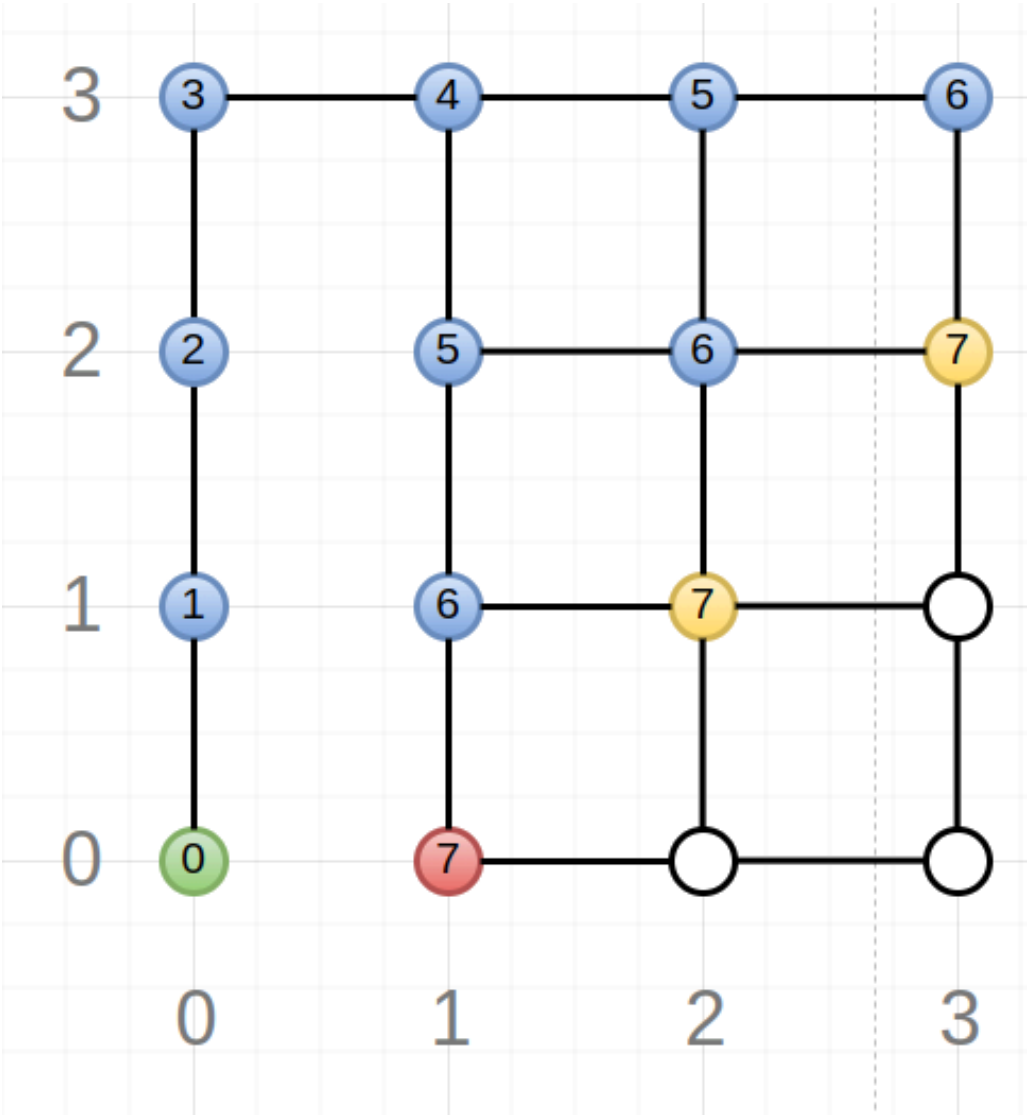


Figure 15: Dijkstra

Algorithme A* - 12

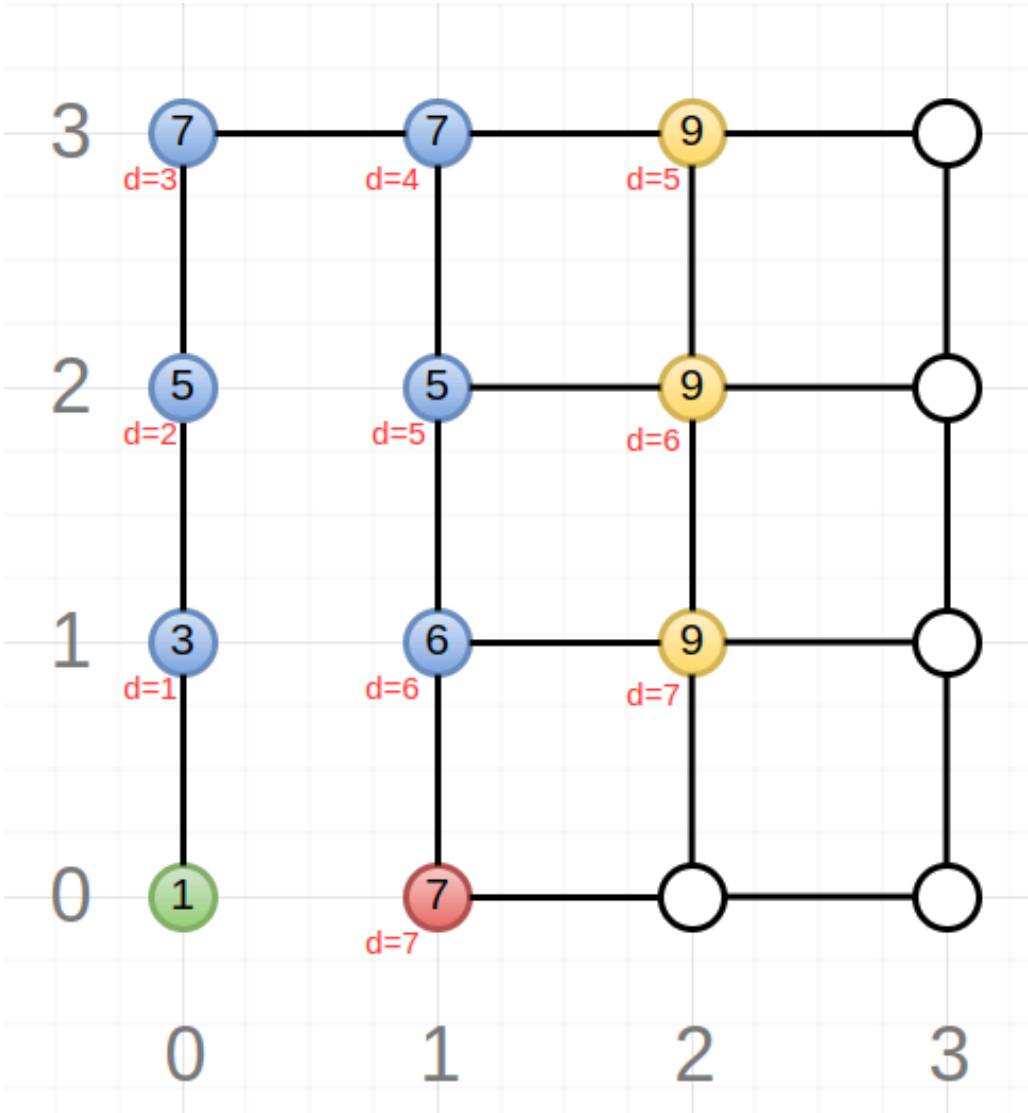


Figure 16: A*

Algorithme A* - 13

Question : Est-ce que la complexité dans le pire des cas de A* est meilleure que celle de Dijkstra ?

Réponse : Non. Dans le pire des cas, la complexité de A* est la même que celle de Dijkstra. Cependant, si l'heuristique est **admissible** et **monotone**, A* explorera au plus le même nombre de sommets que Dijkstra, et en pratique souvent beaucoup moins.

Visualisation

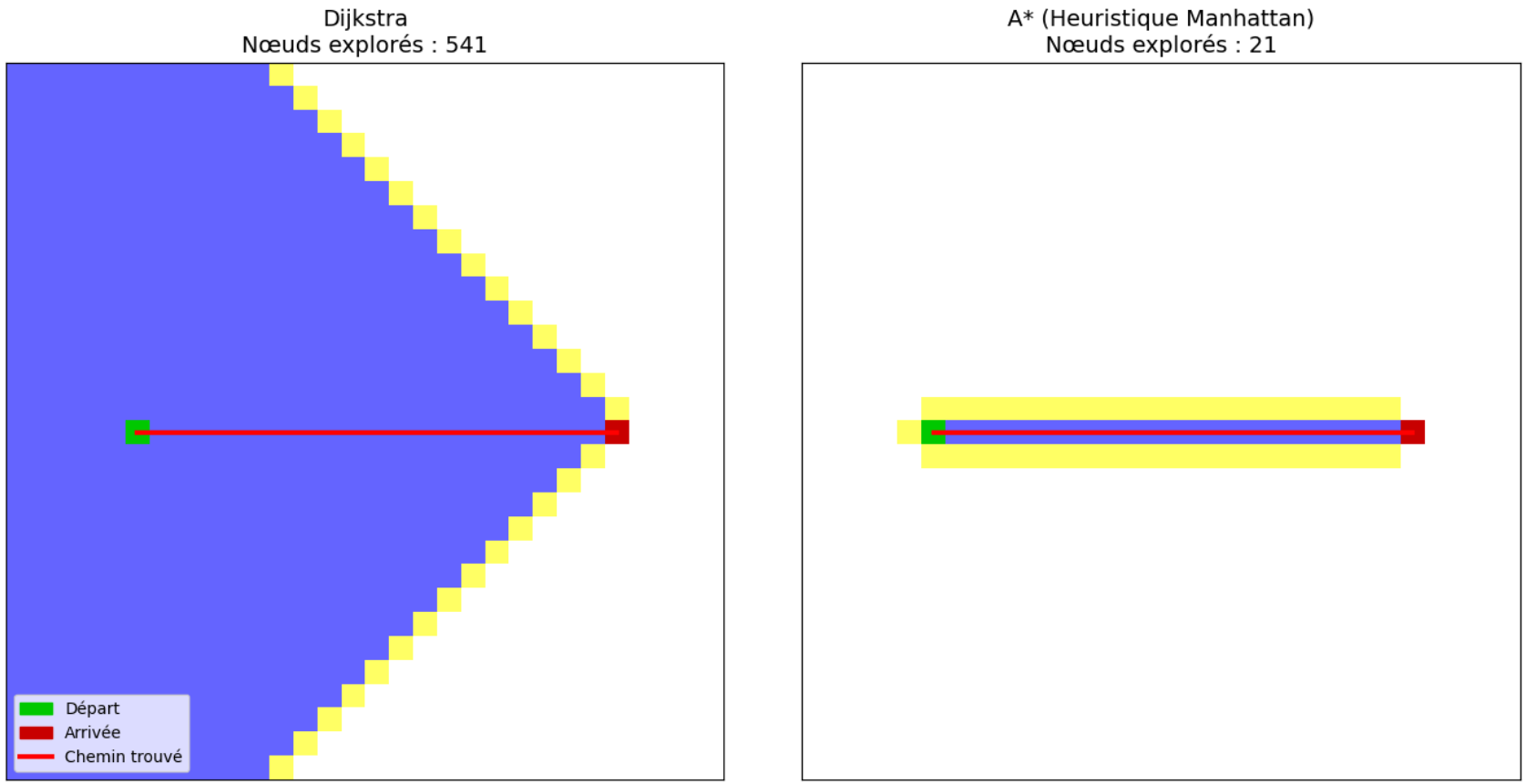


Figure 17: Visualisation de l'algorithme

Visualisation - 2

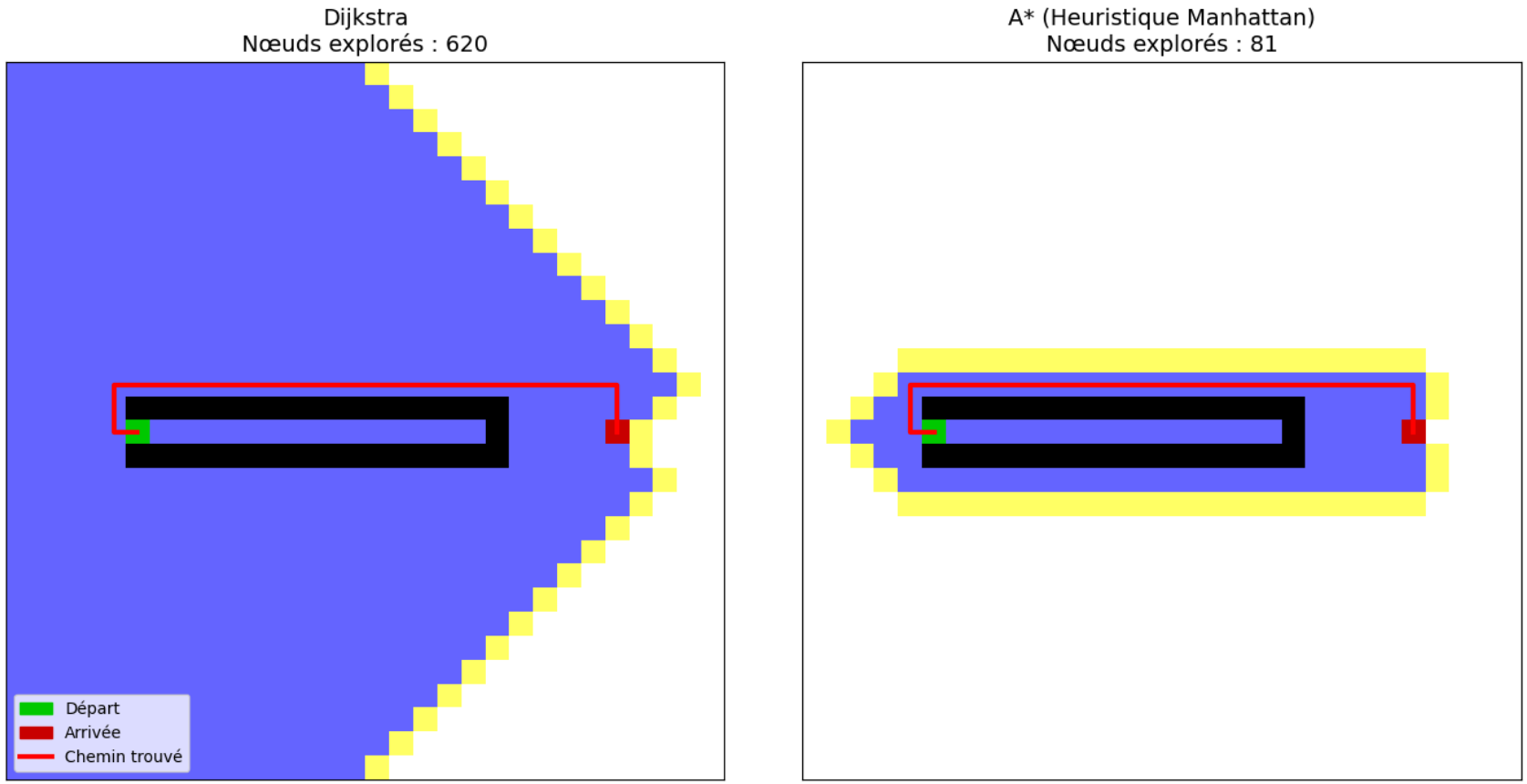


Figure 18: Visualisation de l'algorithme

Visualisation - 3

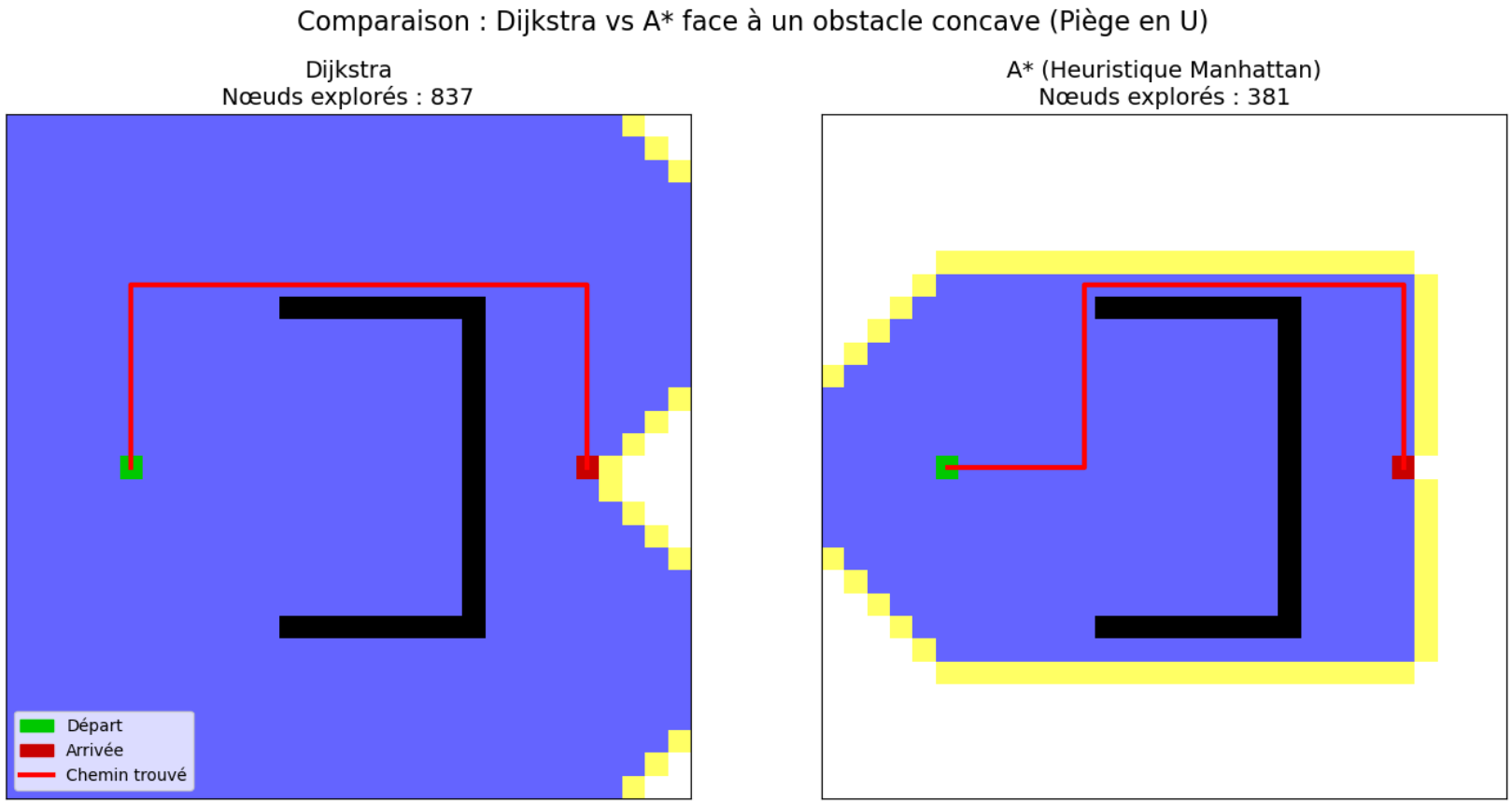


Figure 19: Visualisation de l'algorithme A*

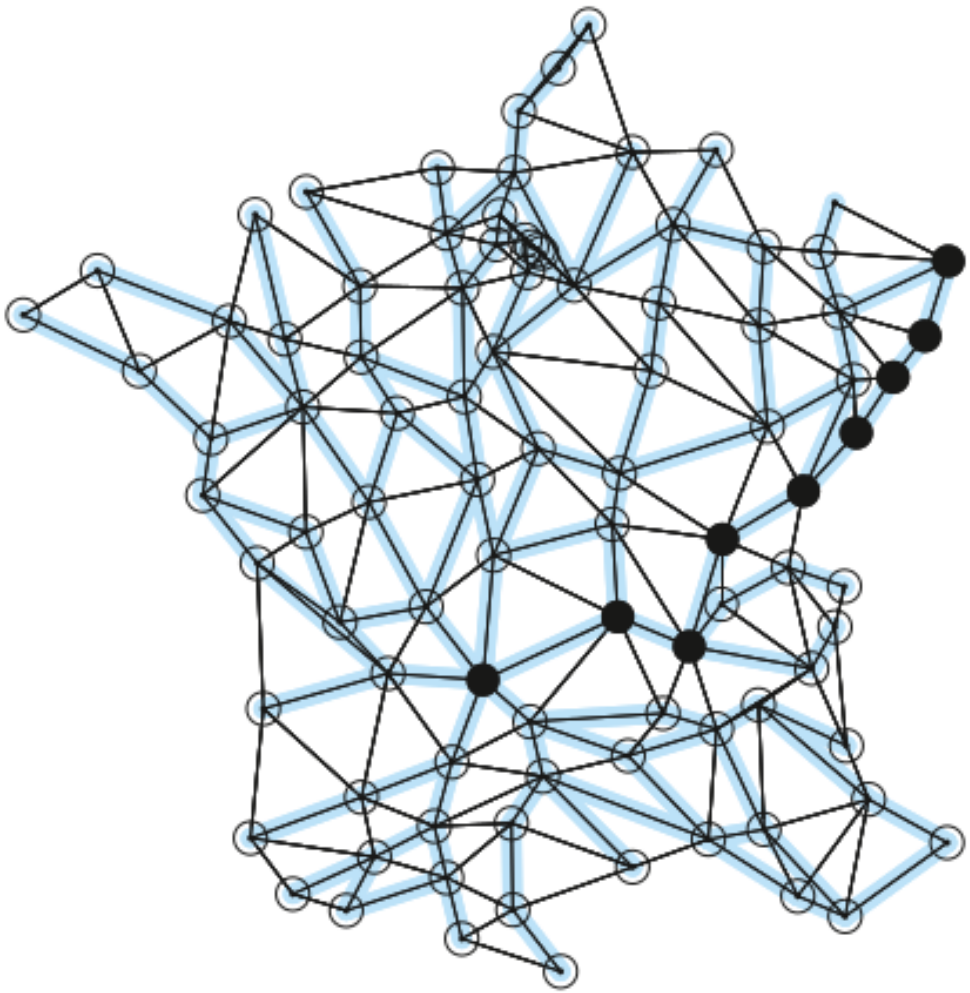
Application : GPS

L'algorithme A* est utilisé dans les systèmes de navigation GPS pour trouver le chemin le plus court entre deux points. (Ex : Google Maps, Waze, ...)

L'heuristique utilisée est la distance à vol d'oiseau (euclidienne).

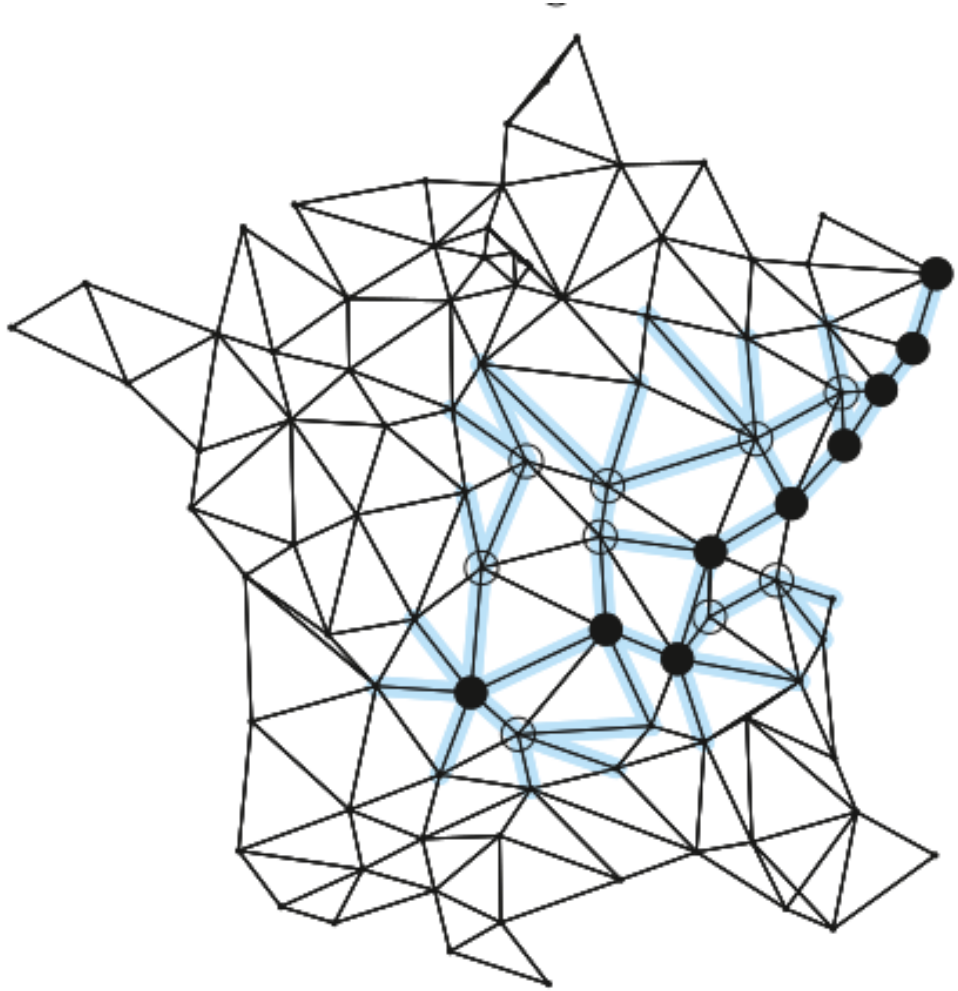
Voici un exemple sur une carte de France, on cherche le chemin le plus court entre la Corrèze et le Bas-Rhin. On compare l'exécution de Dijkstra et de A*.

Application : GPS - 2



Algorithme de Dijkstra :
On a exploré 117 départements de France métropolitaine.

Application : GPS - 3



Algorithme A* :
Seuls 41 départements ont été explorés.